

Intensivtutorium

Programmierung

Semester: Sommersemester 2026

Tutor: Luis Staudt

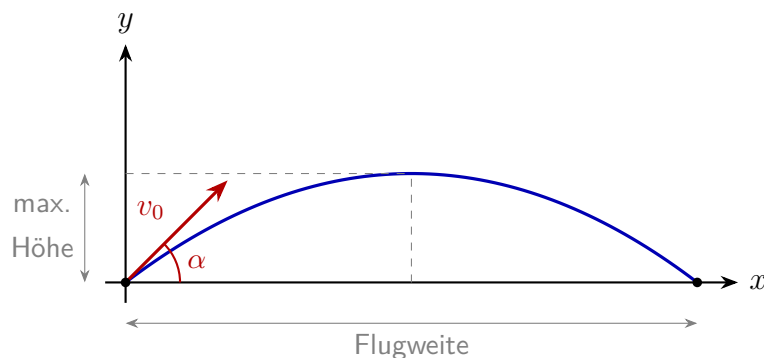
Studiengänge: MIB, OMB, PEB, MVB

Aufgabe 3 – Flugbahn zeichnen

Bei einem schiefen Wurf wird ein Objekt mit der Startgeschwindigkeit v_0 unter dem Winkel α abgeworfen. Wir vernachlässigen den Luftwiderstand. Die Position zum Zeitpunkt t berechnet sich wie folgt:

$$x(t) = v_0 \cdot \cos(\alpha) \cdot t \quad y(t) = v_0 \cdot \sin(\alpha) \cdot t - \frac{1}{2} g t^2$$

mit der Erdbeschleunigung $g = 9,81 \text{ m/s}^2$.

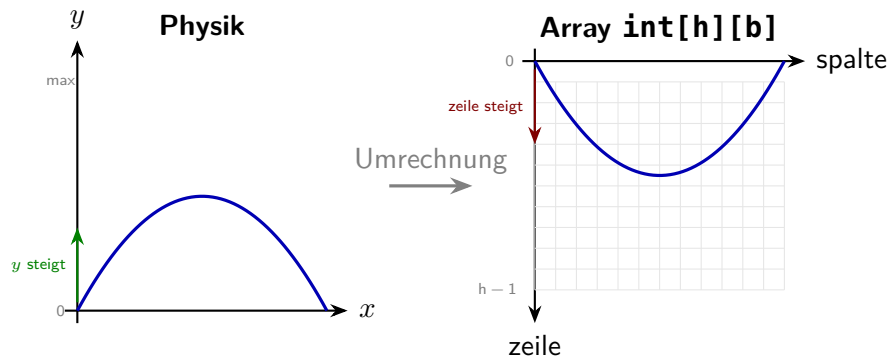


Die Flugbahn soll als Kurve in ein `int[][]`-Raster gezeichnet werden – ähnlich wie bei der Bildverarbeitung. Jede Zelle des Rasters entspricht einem "Pixel". Ein Wert von `0` steht für den Hintergrund, ein Wert von `1` für die Flugbahn.

Koordinatensystem

Das Array `int[hoehe][breite]` stellt das Zeichenfeld dar. **Zeile 0 ist oben**, Zeile `hoehe-1` ist unten (wie bei Bildern). Die y-Achse ist also im Vergleich zum physikalischen Koordinatensystem *invertiert*:

$$\text{zeile} = (\text{hoehe} - 1) - y_{\text{skaliert}}$$



Hilfsfunktionen

In Java lassen sich Winkelfunktionen über `Math.cos()`, `Math.sin()` und die Umrechnung von Grad in Bogenmaß über `Math.toRadians()` nutzen.

Implementiere die folgenden Methoden:

- a) Position berechnen** – Berechnet die Position (x, y) zum Zeitpunkt t und gibt sie als `double[]` mit zwei Einträgen zurück.

```
1 static double[] position(double v0, double alpha, double t)
```

- b) Maximale Flugweite und Höhe bestimmen** – Berechnet durch schrittweises Erhöhen von t (in Schritten von dt) die maximale Höhe und die Flugweite (x-Position beim Aufprall, also wenn $y \leq 0$). Gibt beides als `double[]{flugweite, maxHoehe}` zurück.

```
1 static double[] flugdaten(double v0, double alpha, double dt)
```

- c) Flugbahn ins Raster zeichnen** – Erzeugt ein `int[hoehe][breite]`-Array und zeichnet die Flugbahn als Kurve ein. Die physikalischen Koordinaten müssen dafür auf die Rastergröße skaliert werden. Nutze die Ergebnisse aus `flugdaten()`, um die Skalierungsfaktoren zu bestimmen:

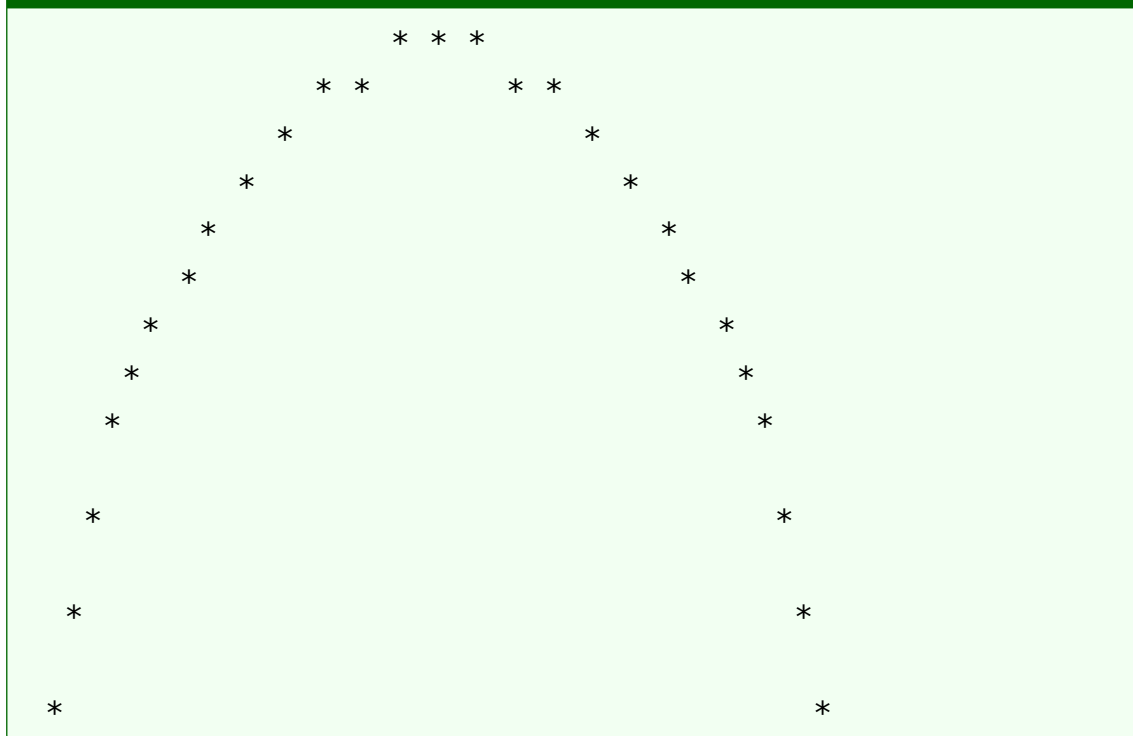
$$\text{spalte} = (\text{int}) \left(\frac{x}{\text{flugweite}} \cdot (\text{breite} - 1) \right) \quad \text{zeile} = (\text{hoehe} - 1) - (\text{int}) \left(\frac{y}{\text{maxHoehe}} \cdot (\text{hoehe} - 1) \right)$$

```
1 static int[][] zeichneFlugbahn(double v0, double alpha,
2                               double dt, int breite, int
                               hoehe)
```

- d) **Raster ausgeben** – Gibt das Raster auf der Konsole aus. Verwende ein Leerzeichen für 0 und ein * für 1.

```
1 static void rasterAusgeben(int[][] raster)
```

Mögliche Ausgabe ($v_0 = 20$, $\alpha = 45^\circ$, Raster 60×20)



- e) **Mehrere Winkel vergleichen** – Zeichne die Flugbahnen für die Winkel 30° , 45° und 60° in *dasselbe* Raster. Verwende unterschiedliche Werte (1, 2, 3) für die verschiedenen Kurven und passe die Ausgabe entsprechend an (z. B. verschiedene Zeichen).

```
1 static int[][] vergleicheWinkel(double v0, double dt,
2                               int breite, int hoehe)
```

Hinweis

Für die gemeinsame Darstellung muss ein einheitlicher Skalierungsfaktor verwendet werden. Bestimme dazu zuerst die maximale Flugweite und die maximale Höhe über *alle* Winkel hinweg und verwende diese als Bezugsgröße für die Skalierung.

- f) **Test in main** – Teste die Methoden mit $v_0 = 20 \text{ m/s}$, $dt = 0,01 \text{ s}$ und einem Raster der Größe 60×20 . Gib die Flugbahn für 45° und anschließend den Vergleich der drei Winkel aus.